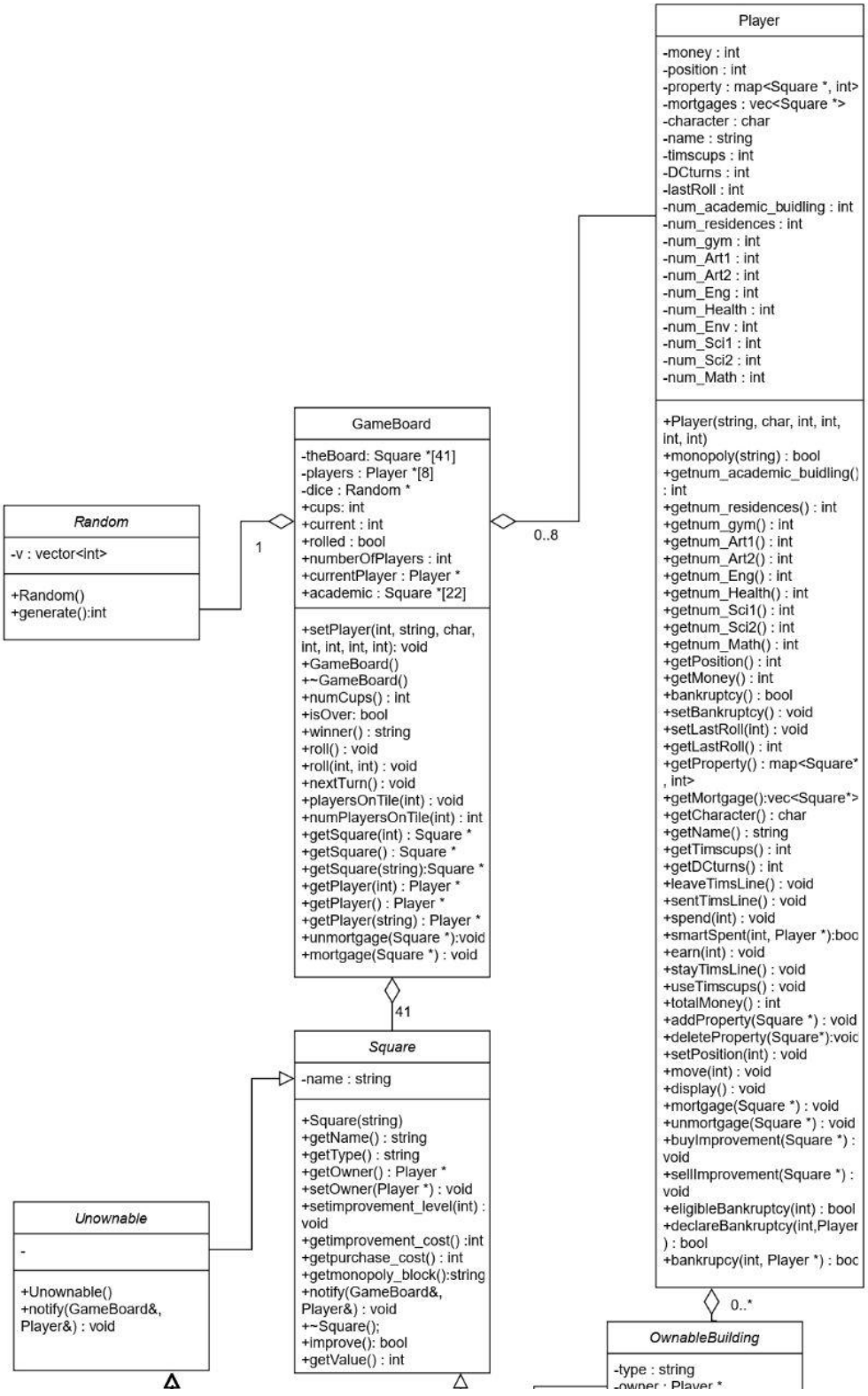
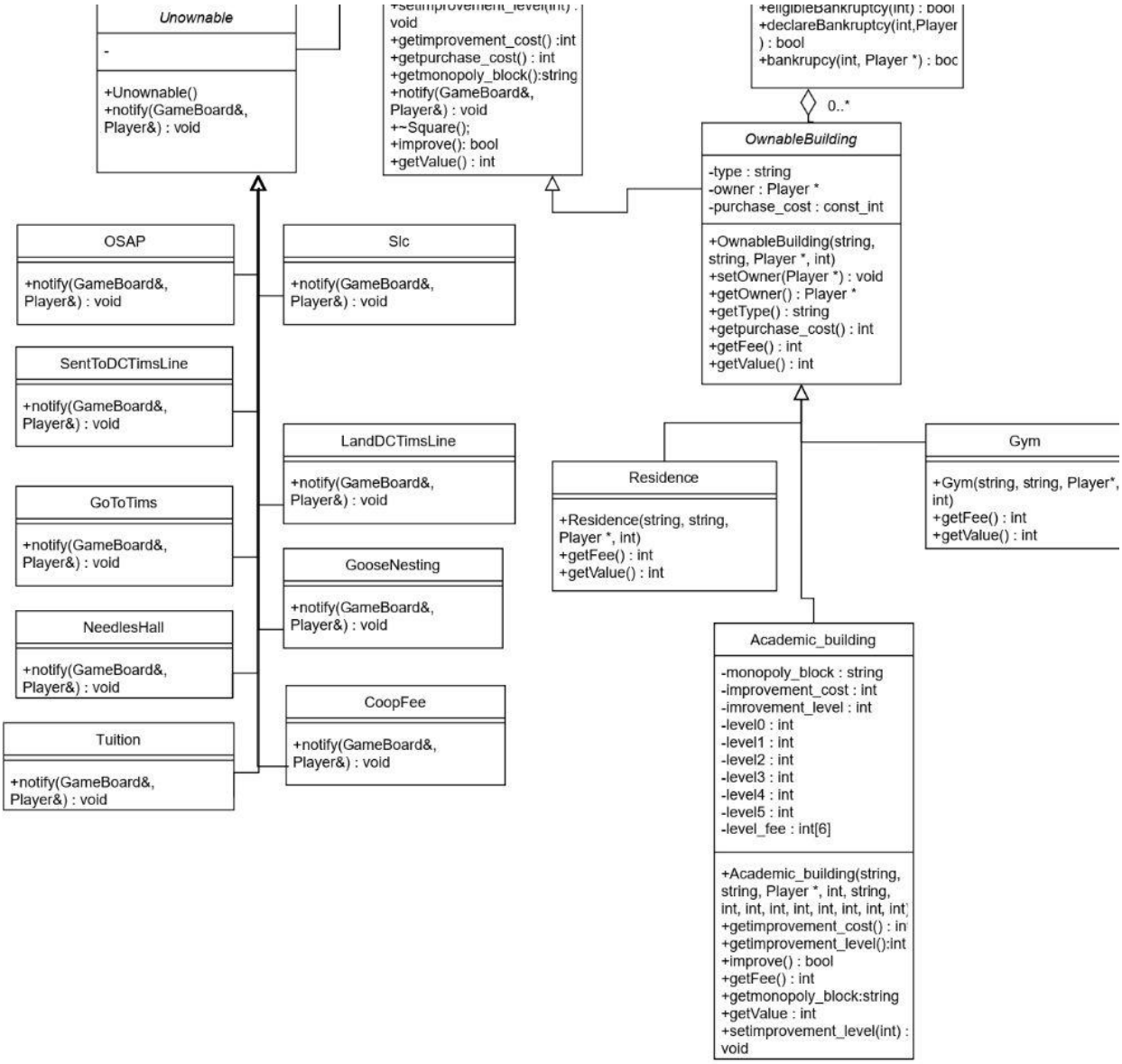
A5 - Watopoly Design

Brian Liu, Alwin Chen, Qirui Liu

UML (also included as a pdf)





Overview

Our overall program consists of three main classes: GameBoard, Square, and Player as well as a main.cc file. Two classes inherit directly from abstract class Square: OwnableBuilding and Unownable, each with their own subclasses which act like the individual squares on a Monopoly board. GameBoard and main.cc handle game logic, such as controlling the flow of the turns and player order as well as other functions such as mortgages and auctions. All subclasses of Square will notify the GameBoard when a player lands on them, and will perform different operations depending on if they are an academic building, residence, gym, etc. Player contains all the data related to a specific player in the game, as well as providing methods that only involve one player, such as buying properties and moving across the board.

Design

The main design pattern we employed to solve the challenges of this project was an observer pattern for the squares of the monopoly board. In the case of our code, the abstract subject is the Square class (square.h) and all classes inheriting from Square are the concrete subjects. The GameBoard class is the concrete observer. We decided to omit the abstract observer class from our design as we only have GameBoard as a concrete observer (gameboard.h). We also decided to make our text display a helper function rather than an observer of the gameboard like in A4Q3. The reasoning is that if we chose a similar approach then we would need to initialize and manage a 2-D array with side lengths of more than 90, which would be very tiresome and costly to manage. Instead, our text display (textdisplay.h) instead is passed a pointer to a GameBoard and prints required information to stdout.

The GameBoard class "observes" an array of Squares, and when a player lands on a Square, the notify function is called, which performs different actions based on which Square is notifying. The notify function will differ when called by different subclasses to match the intended behavior when a player lands on one of those types of Squares. The GameBoard class also owns a "Random" class, which has member functions that allow us to simulate dice.

Our main function (main.cc) also owns one GameBoard, and based on commands received from the user via stdin, will produce appropriate output to stdout and call appropriate member functions from GameBoard based on what command has been inputted. Our main function also allows us to load save games using a command line argument as well as providing a command to the user to save a game state to a chosen destination file. Our main function also provides users with the option to enter "testing" mode using a command line argument. Testing mode provides users with the option to simulate rolls using an overloaded roll function. This function allows users to choose any two integers (even negative) to move their player around the board, which is very useful for testing purposes.

Resilience to changes

We designed our classes, member functions, and main with high cohesion and low coupling in mind.

For cohesion, our goal was to have our modules doing the following tasks and nothing else:

- GameBoard: manages interactions that involve multiple players or involve players and properties like mortgaging, auctions, and rolling the dice (but not actually updating their position)

- Player: manages player specific interactions that can be performed successfully without needing access to other players, such as movement, buying buildings, and buying improvements.
- Square: manages what happens when a player lands on them as well as containing data about properties such as their owner, improvement costs, and how many improvements they have.
- Main function: controls game logic and provides users with options such as loading and saving files as well as a testing mode.

All of our modules contain functions that operate together to work towards completing one task. Because of this, our program has high cohesion.

For coupling, our modules are limited to communicating with each other using functions. There are a couple of member functions that modify values in another class, such as our auction function, which is given and modifies an integer reference as a parameter representing the highest bid. However, overall, our program aims to minimize coupling and was successful in achieving somewhat low coupling with a few areas with higher coupling.

Because of this, our program is very adaptable to changes in the program specifications and would require not too much work to change. For instance, if we wanted to modify our SLC and Needles Hall squares to be more like Chance and Community Chest from actual Monopoly, we would just have to modify a single function in their respective classes to modify behavior when a player lands on them. All other aspects of our code would remain the same. Another example would be if we wanted to implement "house rules" and different game logic, then we would only need to make appropriate changes to our main function as our main function manages all game logic. No other modules would need to be changed.

<u>Deviation from plan of attack on due date 1</u>

We stuck to the time schedule quite perfectly.

Deviation of player class:

The deviation from the UML on due date 1 is significant because we underestimated the complexity of the Player class. We first thought to store all the property information in the academic_buliding class but it turns out that maybe having a copy of the data inside the Player class would be much easier for calculation of bankruptcy. So as a result, we added related information inside of the Player class and getter/setter methods respectively. As for the bankruptcy, we didn't want to put it in Player class directly because there is a command taking in "bankrupt". However, since bankruptcy is more of a player-characteristic feature, we finally

decided to put it back to Player class. Also we add some other functions that are related to bankruptcy such as eligibleBankruptcy() to check is a player is eligible to declare bankruptcy.

Deviation of Unownable class and Square class:

There is not much deviation from the plan except a few public virtual methods added in order for the program to compile and run.

Deviation of Ownable class:

Rather than before, I decided to edit the ownable class with improvement_level for all subclasses, because we need to decide whether the ownable building is mortgaged. (We assign improvement level is -1 for all mortgaged classes!) In addition, because I forgot to connect with other classes, thanks to teammates, we added a notify function passing the reference of the board and the player to pay the fee and to do an auction!

Deviation of main function:

Generally, the basic property is the same as before, except we add a “newgame” command for starting a new game. In addition, based on the work of friends, we finish almost all commands of the game.

Deviation of GameBoard:

The function and design of GameBoard remained essentially the same as what we had originally planned. The main changes that we ended up making were adding and removing some member functions as well as modifying some of the member functions we had originally planned.

Deviation of text display:

Originally, we wanted our text display to be an observer of our gameboard and to have an internal array that was updated whenever there was an update to the gameboard. However, this would’ve been very tedious, so we chose to make it a helper function that is passed a pointer to a gameboard and then printed required info instead.

<u>Answers to questions</u>

After reading this subsection, would the Observer Pattern be a good pattern to use when implementing a game board? Why or why not?

We still stick to our previous answer to this question that Observer Pattern would be a good pattern for gameboard since moving pieces (characters) will trigger the function of the squares on the board and change the output, but what deviates from the standard Observer Pattern discussed in class is we didn't set subject class and observer class, we implement a helper function called printBoard to do printing work.Although the monopoly is quite similar to Reversi that they both need to change the board once receiving commands, the difference is that printBoard in Watopoly is the only observer so a helper function will be enough.

Question. Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?

Although we didn't implement this extra credit, we think the Factory method pattern may be a good design pattern since we want to generate polymorphic objects based on (maybe) complicated policy decisions. Different cards have different probability and may/may not be put back. Different cards can be implemented via polymorphism and just as the enemy example in class, different cards have different effects (methods). The superclass's effect method is implemented as pure virtual and each subclass's effect method overrides.

Question. Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?

This question deviated from our previous answer that we should use Decorator Pattern, though Decorator pattern is a good pattern for this problem. We instead use int arrays to store the rent calculation paid to the owner under different improvements. As for the complexity, we think these two are roughly the same since each academic building has its own improvement costs and no obvious rules can be seen, so we have to manually initialize those. For the sell improvement action, we think int arrays are better since decorator patterns cannot eliminate more than one improvement at one time, but int arrays can do this by changing improvment_level variable in academic building class.

Extra credit features

Apart from the given commands to be implemented, we add an additional command "newgame" that delete the old game and restart from the beginning. This includes key in new players with their name and characters and start the game.

Final questions

What lessons did this project teach you about developing software in teams?

The first lesson is that working as a team requires more than knowledge. Different teammates have different time schedules and sometimes it is hard to arrange a group meeting. In this case we perhaps need more time to finish the group project.

The second lesson is that formatting and naming is more important than we thought. Working in a group requires us to interact with each other's codes, this inevitably causes some formatting and naming issues. Spending hours on just regulating the naming patterns is bad.

The third lesson is that the plan of attack should be as specific as possible and resilient to change. Life happens and we must prepare in advance. There were lots of problems due to the miscommunication between teammates, for example, before we actually started writing codes, we all agreed to write and upload our .h files first. However, .h file changed and the user of this class need to react accordingly, resulting in a chain reaction.

The last lesson is commenting. A lot of problems also rises from misunderstanding, for example, the user of a class member function may not know what is happening inside and cannot judge from the name of that method neither.

What would you have done differently if you had the chance to start over?

If we could have the chance to start over, we first want to make a detailed plan of attack at the very first beginning of our project before we actually work on it. We would go through all the classes and methods together from base level to the upper-most level to see if any methods are necessary. This may be hard for a single person to come out because each single person may not have a high level of view. Then as we did successfully, write all the .h files first and comment on any methods and classes that may be ambiguous. Finally, leave a few days before the project is due in case of emergency. And finally, try to do oneself's coding part as best as possible.

Conclusion

Spending hours writing thousands of lines of codes as a group is suffering, but no pain no gain. It is unavoidable to have a team project in the future or having a giant project that needs to be done in a few weeks. This project not only told us how to cooperate with each other but also enhanced

our understanding of the course content, sometimes even trying to compile all the codes is challenging. But in word, a beneficial course and a very educational assignment!